MARS: A Memory-Augmented Agentic Terminal Multiplexer that Learns From Your Commands

Anjishnu Kumar*
Microsoft Copilot AI
Redmond, WA
anjikumar@microsoft.com

Abstract

This paper presents MARS, a terminal editor, multiplexer, and LLM agent that ship as a single binary. Because the agent is embedded where the user works, it has access to two sources of context that a cloud assistant cannot reach: the user's own history of commands, and the tool's own documentation and configuration registry. MARS retrieves from both with plain lexical search (BM25 over local files, with no embeddings and no data leaving the machine) and injects the results into the model's prompt. In the constrained domain of the terminal this simple mechanism is unusually effective. Under a leakage-controlled protocol in which the memory store holds only *paraphrases* of the user's earlier requests, retrieval raises the accuracy of a small, inexpensive model (claude-haiku-4-5) on project-specific commands from 38% to 96%, so a win requires the retriever to bridge a paraphrase gap rather than look up a planted answer. Pointed at the tool's own documentation, the same mechanism raises self-knowledge accuracy from 43% to 76%, and accuracy at mapping a plain-English request to the correct configuration setting from 17% to 93%. We describe the directive protocol that grounds the agent in the editor's action registry, the corrective-memory loop, the architecture that keeps the agent and the user's API key available across machines, and a self-instrumenting evaluation built from the agent's own debug log.

1 Introduction

LLM assistants have become a routine part of terminal work. A typical user, an ML researcher training models across a laptop and a handful of remote GPU machines, babysits long runs in a shell, edits configs, holds sessions open with a multiplexer, and asks an assistant to explain a CUDA out-of-memory trace or draft an `rsync` command. In practice the assistant is the weakest link, for two reasons that compound. It is generic: it runs in a separate window, sees only the fragment the user pastes into it, and does not know that in this project a training run is launched with a particular `sbatch` invocation settled on last week. It is also amnesiac: it forgets the command the user corrected yesterday, forgets "where was I" on reattach, and knows nothing about the editor it sits beside. In the remote setting that defines GPU work there is a third problem: reaching the work at all usually means exporting an API key onto a machine the user does not own, where it lingers in the environment, the shell history, and often a dotfile.

Our observation is that an agent embedded at the terminal has direct access to memory that a browser-tab assistant never sees. Two corpora sit within reach of a terminal-resident process: the user's own history of commands they actually ran, and the tool's own documentation, action registry, and configuration schema. Neither corpus is large, and neither requires training. What they require is an agent that lives where they live, persists between requests, and captures corrections as they happen; per-invocation CLI agents (Anthropic, 2025) satisfy the first condition but not the other two (§2). We built MARS (a Memory-Augmented Rust System) to satisfy all three: it retrieves from both corpora with plain lexical search and injects the results into the prompt.

We expected local memory to help. We did not expect a method this simple to nearly close the gap between a small model and its personalized ceiling, even when the stored memory is only a *paraphrase* of the user's earlier request. On a set of project-specific commands, retrieving a handful of reworded past corrections lifts claude-haiku-4-5 from 38% to 96% accuracy under a protocol in which a win requires generalizing across a paraphrase gap, not looking up a planted answer. In a

* Work done before joining Microsoft.

narrow domain, memory does not need to be sophisticated to be decisive.

This paper makes four contributions. Firstly, a memory-augmented terminal agent (§3) built from minimal parts: a registry-validated directive protocol, a corrective-memory loop, and a model cascade. Secondly, two axes of *local* memory: the user's own commands for translation, and the tool's own ground truth for self-knowledge and reconfiguration. Thirdly, the architecture (§4) behind always-present local memory: a persistent daemon, and an SSH broker that keeps the API key off remote machines. Finally, a self-instrumenting evaluation (§5) in which corrective memory takes a small model from 38% to 96% on personalized commands under a leakage-controlled protocol. MARS is open source under the MIT license (Kumar, 2026), installs with `cargo install mars-terminal`, and is about 9,500 lines of Rust; the gold sets, seeded memory stores, and frozen result files ship with the source.

2 Related Work

Terminal tooling. Multiplexers such as tmux (Marriott and tmux contributors, 2024) and Zellij (Zellij contributors, 2024) provide persistence but no intelligence. Atuin (Huxtable and Atuin contributors, 2024) makes the user's shell history a first-class searchable database, and thefuck (Iakovlev and contributors, 2024) repairs a mistyped command from its error output; both share our premise that the user's own history is valuable, but neither feeds it to a model. AI terminals such as Warp (Warp, 2024) add command search but see only their own blocks. CLI agents such as Claude Code (Anthropic, 2025) and GitHub Copilot in the CLI (GitHub, 2024) put a cloud model in the shell and can read the same local files MARS reads; however, they run per invocation, so nothing persists between calls, corrections are not captured as they happen, and every remote machine needs its own credential. IDE agents such as Cursor (Anysphere, 2024) lose context when the window closes. MARS differs in the combination: one persistent process that hosts the editor, captures every accepted correction, grounds the agent in its own action registry, and keeps the credential home.

Natural language to commands. NL2Bash (Lin et al., 2018) and Tellina (Lin et al., 2017) established the NL-to-shell task and its corpora; we build on the task rather than the models, personalizing translation with the user's own history.

Agents and retrieval. Directive-style acting relates to ReAct (Yao et al., 2023) and tool use to Toolformer (Schick et al., 2023), though we deliberately use a portable text protocol with registry grounding rather than provider-native tool calls. Our memory is retrieval-augmented generation (Lewis et al., 2020) in its simplest lexical form; the novelty is the setting, retrieval over the user's *own* local context, embedded where it is produced.

Computer-use agents. A parallel line of work drives whole desktops rather than terminals. Anthropic's computer use (Anthropic, 2024) operates GUIs from screenshots, OSWorld (Xie et al., 2024) benchmarks such agents on real operating systems, OS-Atlas (Wu et al., 2025) pretrains a foundation action model for GUI grounding, and Plan-and-Act (Erdogan et al., 2025) separates a planner from an executor to survive long-horizon tasks. The terminal offers the same ambitions a simpler substrate: the action space is already text, so grounding is a registry lookup rather than a vision problem, and a plan is a command sequence the user can read. These techniques map onto the rungs above single-command recall (§6): procedure mining is a planning problem, and workflow understanding is the terminal's analogue of GUI-state tracking.

3 The Memory-Augmented Agent

3.1 Design considerations

The agent should be another input device, not a second system. The recurring risk in an agentic editor is that the agent becomes a subsystem bolted beside the editor, with its own command language, state, and failure modes. We chose the opposite design. Everything MARS can do is a variant of a single `Action` enum, and three interfaces resolve a user's intent to it: a direct keychord (procedural recall), a fuzzy command bar (recognition), and the agent (natural language). All three terminate in a single dispatch point, so a capability added once is chord-bindable, searchable, and agent-invokable. Figure 1 sketches the retrieval loop.

This choice trades expressive freedom for grounding. Because the agent can only name actions that exist in the registry, and retrieves from flat local files rather than a vector index, the sur-

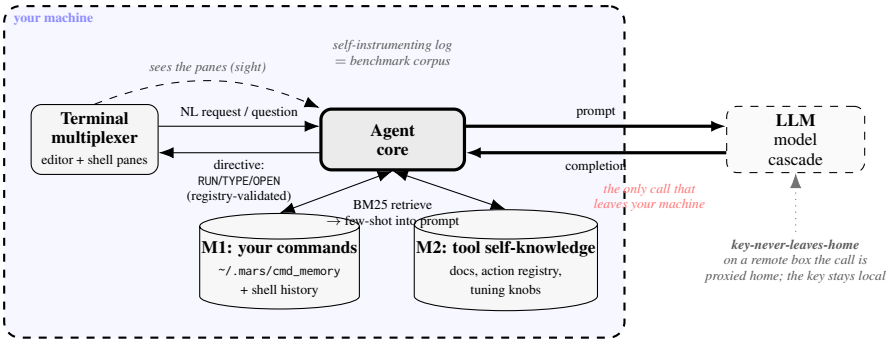

Figure 1: MARS augments a terminal LLM agent with local memory over the user's own context. Seeing the multiplexer's panes, the agent retrieves with lexical BM25 from two local stores (M1, the user's past commands, and M2, the tool's own docs and registry) and injects the results as few-shot exemplars. Only the model call leaves the machine; on a remote box even the key stays home, and every call is logged.

Directive	Effect (gated by confirm)
`RUN: <Action>`	fire a registry action
`TYPE: <cmd>`	type a command into a shell pane
`OPEN: path:line`	jump to a file location
`NEED: <context>`	pull more context; re-ask once

Table 1: The directive protocol. A `RUN:` naming an action absent from the live registry is rejected before it fires, so a hallucinated action cannot execute; a destructive action passes the same confirmation gate a human would.

face a user must trust is small and easy to audit. However, MARS cannot invent a capability at runtime, and lexical retrieval will miss paraphrases that an embedding model would catch (§5). In the terminal domain we believe this trade is strongly favorable.

3.2 The agent as a validated input device

An agent that acts on the user's workspace must be safer than a keypress, never a bypass around the editor's gates. Following the mode-error tradition in interface design (Norman, 1988), we treat the agent as one input device among several, behind the same confirmation gates as a keystroke. The implementation is a trailing-line *directive* protocol: the model receives a live catalog of every action, description, and keybinding, and ends its reply with at most one directive (Table 1). We chose a plain-text protocol over provider-native tool-calling APIs because it works identically across OpenAI-compatible endpoints, a native Anthropic path, and local open models, and because it renders as human-readable text that the user confirms before anything fires; the cost is that we give up the stricter typing a schema provides.

The grounding comes from the registry check. A `RUN:` whose name does not resolve against the live `Action` set is rejected before dispatch, so the model cannot execute a hallucinated capability. The `NEED:` directive closes a different gap. Rather than sending the full screen with every request, the model can ask for more context (a pane's scrollback, another tab) on demand; MARS supplies it and re-asks exactly once, so no retrieval loop can form.

3.3 Natural-language shell translation

The most frequent agentic request at a terminal is to turn English into the right command. *Translation should be assistive and reversible, never an automatic execution.* The implementation lives on the command bar: the user presses `Ctrl+Space` and types plain English ("find the biggest checkpoints here"); MARS sends the request with on-screen context (working directory, recent output) to the model cascade (§3.7) and renders the proposed command at the cursor to run, edit, or discard. The same translator also serves as the command bar's fallback: plain text that matches no registry action is treated as a natural-language command request (§4). Translation by itself is a commodity capability (Lin et al., 2018, 2017); what personalizes it is the memory we describe next.

3.4 Memory 1: the user's own commands

The user's own accepted commands are the highest-value context available, and they are local to the terminal by construction. A generic model does not know the user's commands. Asked to "launch the training run," it produces a plausible `python train.py` when this project in fact uses `sbatch -gres=gpu:8 scripts/train.slurm`.

The implementation is a corrective loop. Every time the user accepts (or edits and runs) a translated command, MARS appends the `(request, command)` pair to a local JSON-lines file, and it also reads recent shell history from `$HISTFILE`. On the next request, MARS ranks these entries against the query with BM25 (Robertson and Zaragoza, 2009), with the standard parameters k_1=1.5 and b=0.75 and no index to build, and injects the top three pairs as few-shot exemplars, together with up to three ranked lines of recent shell history.

The loop is corrective in the precise sense that a command the model got wrong once becomes one it gets right thereafter, because the user's correction is what enters the store; a per-invocation assistant never observes the correction. Nothing in this loop is novel as information retrieval; the contribution is the observation that in this domain the retrieval can be this simple and still be decisive (§5). The cost is a bounded prompt inflation of a few exemplars per request, which §5 measures and finds small relative to the accuracy gain.

3.5 Memory 2: the tool's own documentation

A tool's own capability surface is authoritative documentation and cannot drift from the code. An agent embedded in a tool should answer questions about that tool, and change its settings, from ground truth rather than a guess. MARS's action registry, keybindings, and tuning knobs each carry a human-written description, so documentation is generated from the code itself. The same BM25 retriever runs over this second corpus: documentation chunked into passages, plus the live registry and the described knobs. On a `NEED: docs` pull, MARS injects the top passages, so "how do I split the screen?" is answered with the real keybinding and "make the accent orange" resolves to the actual knob and a confirmable edit. When retrieval returns nothing, the agent reports that the capability does not exist rather than inventing one.

3.6 Why simple memory is enough

It is worth stating why a method this simple suffices, because the instinct when building a memory system is to reach for embeddings and a vector database. The terminal is a constrained domain, and the constraint favors lexical retrieval in three ways. Firstly, a user's command vocabulary is small, repetitive, and lexically stable: the same flags, paths, and tool names recur verbatim, so a query and its useful memory tend to share surface tokens, exactly the regime in which BM25 is strong and dense retrieval's paraphrase advantage is marginal. Secondly, the corpora are small, on the order of hundreds of entries, so there is no scaling pressure to justify an index. Thirdly, the payoff is asymmetric: a single relevant exemplar of the user's own command is worth more than semantic breadth, because it supplies the exact string the user wants reproduced. We therefore treat embeddings as a cost to be deferred until the domain broadens (§6). The empirical support is the corrective-memory result of §5, in which BM25 bridges a mean word-Jaccard gap of 0.36 with nothing more than a prompt, a local file, and a standard ranking function.

3.7 The model cascade

Model choice should be right-sized per task. Most LLM traffic at a terminal is cheap and can be served by small or free models, but a fixed model wastes quality where it is not needed and hits rate limits where volume is high. We distinguish two orthogonal reasons to change models. Rotating *for limits* moves a task horizontally, across models of the same tier, when a rate-limit error arrives; because each provider meters usage on its own counter, rotating across N families multiplies the effective free ceiling by roughly N. Escalating *for quality* moves a task vertically to a stronger tier, and only on a detected failure. Detection is free because the agent's output is structured: a `RUN:` that fails the registry check is an unambiguous signal that a small model erred, and it triggers one retry at a higher tier. The cascade costs a little dispatch complexity; the benefit is headroom against the provider quota walls we later hit during evaluation (§5.1).

4 System: The Substrate for Local Memory

Local memory is only useful if the agent is reliably present where the memory lives, across disconnects and on machines the user does not own. The architecture exists to guarantee that presence.

One binary, one core. MARS is a single Rust binary whose application core owns all state (buffers, panes, terminals, the agent conversation, the memory stores) and never touches a TTY: input arrives as source-neutral events, and rendering is a stateless projection onto a `ratatui` backend.

The same core runs unmodified as a standalone terminal, a headless daemon, and a test backend, which lets the evaluation drive the real system with no mocks (§5).

A persistent daemon. We implemented persistence as a process split rather than as a save-and-restore feature. A server process runs the entire application headless and emits the editor's own ANSI byte stream over a Unix socket; a thin client owns the real terminal and pumps bytes. We chose to have the server render ANSI rather than define a structured wire protocol because it reuses the entire rendering pipeline; the cost is a wire that carries rendered frames rather than semantic events. The benefit for memory is direct: because the core keeps running with no client attached, the command store and the user's session survive a closed laptop, and an overnight run can be watched and summarized for the user's return.

Classifier-free routing. The command bar routes on a *sigil* rather than a learned classifier: the first character of the input names the target namespace. `!` runs a shell command, `?` asks the agent, `@` opens the file navigator, and plain text fuzzy-searches the action registry. Plain text that matches nothing in the registry falls back to the agent: the bar treats it as a natural-language command request and hands it to the translator of §3.3, whose proposal the user confirms before it runs. Routing remains deterministic under the fallback, triggered by a registry miss rather than a classifier score, so there is no routing model to train, evaluate, or explain.

The key never leaves home. *A secret is a fact about the user, not the machine, and should live in one place.* The implementation is a broker. A home-side daemon (`mars keyd`) holds the key; when the user runs `mars ssh`, MARS reverse-forwards the broker's socket over the SSH connection, and the agent on the remote box makes its model calls by proxying each request home, where the broker injects the credential and streams the completion back. The key never lands on the remote machine, on disk, in the environment, or in memory, so a compromised box has nothing to exfiltrate. The remaining exposure is the open tunnel: a compromised box can use the credential while the tunnel is up, though it cannot steal it, and access ends the moment the tunnel closes.

The log is the benchmark. Running with `-llm-debug` appends one JSON line per model call (task, model, real token counts, latency, and the full prompt and reply) to a local stream, and records whether each suggestion was accepted, edited, or rejected. The log is both a cost profile the user can inspect and, as §5 shows, the source of every prompt, context, and token count the evaluation reports: the system instruments itself.

5 Evaluation

5.1 Setup

We evaluate the two memory axes on frozen gold sets, fixed before any system was run against them, and a single small model under test: 112 command-translation items (88 general, 24 project-specific) and 82 self-knowledge question–answer items (52 knowledge, 30 reconfiguration). The model under test is claude-haiku-4-5, a small, inexpensive model. The thesis requires the model to be *small*, so that personalization has a gap to close, but not necessarily free: both free tiers we tried first (Qwen3-32B on Groq, Gemini Flash-Lite) exhausted their daily token quotas partway through the roughly 350-call batch and began returning empty completions (the quota wall the cascade of §3.7 routes around), so we ran the final numbers on a small paid model. Across all reported runs there were zero empty completions.

Correctness is scored by a stronger judge, claude-sonnet-5 (Zheng et al., 2023), so that the model under test does not grade itself; judge and testee share a model family, a limitation noted in the Limitations section. The corrective-memory store is seeded per condition from paraphrases generated by a separate LLM pass and frozen alongside the gold sets; during the experiment it contains the 24 seeded pairs and no distractors, and retrieval injects the top three (§3.4). Prompts, contexts, and token counts come from the self-instrumenting log of §4.

5.2 Axis A: learning the user's commands

The base numbers in Table 2 confirm the premise of the paper. The small model is already a competent general shell author at 90%, but it falls to 42% on project-specific commands it cannot know, which drags overall accuracy to 79%. These are missing-knowledge failures, not reasoning failures, and they are exactly what local memory targets.

Setting	Accuracy
General (n=88)	90% (79/88)
Personalized (n=24)	42% (10/24)
Overall (n=112)	79% (89/112)
Corrective memory (personalized, n=24, leakage-controlled)	
Cold (no memory)	38% (9/24)
Paraphrase memory	**96% (23/24)**
Verbatim memory (ceiling)	100% (24/24)

Table 2: Command translation (claude-haiku-4-5, judged by claude-sonnet-5). The corrective-memory block is leakage-controlled: the store holds only a *paraphrase* of each prior request and the model is tested on the original, so 96% measures generalization; verbatim memory is the lookup ceiling. Base and cold come from separate runs; they differ by one borderline item (Appendix A).

The corrective-memory experiment isolates that fix, under a protocol designed to rule out the objection that memory is mere lookup. For each of the 24 tasks the store is seeded not with the prior request verbatim but with an independently generated *paraphrase* (mean word-level Jaccard 0.36 to the test query), while the model is tested on the *original* request. A warm win therefore requires bridging the paraphrase gap, not copying a planted answer. Under this setup, cold accuracy is 38% (9/24) and paraphrase memory raises it to 96% (23/24). Fifteen items flip from wrong-or-partial to correct, and one item regresses, from correct cold to wrong with memory; the appendix examines both directions. A verbatim-memory condition, with the exact prior request in the store, reaches the lookup ceiling of 100% (24/24). The headline is the paraphrase number: simple BM25 over a local file recovers almost all of the personalization gap by generalizing, not by lookup. Memory costs about 66 extra tokens per call, and the sign of that cost is model-dependent (Appendix A).

5.3 Axis B: learning the tool itself

The second axis asks whether documentation memory lets the agent answer questions about MARS and reconfigure it. Retrieved documentation lifts overall accuracy from 43% to 76% (Table 3), but the gain is unevenly distributed. On "how-do-I" knowledge questions the gain is modest, from 58% to 65%, because the no-memory model already answers many correctly, and even with docs it sometimes hallucinates a keybinding the docs state exactly (Appendix A). The decisive gain is on *reconfiguration*, turning a request like "autosave more often" into a concrete knob edit, which rises from 17% to 93%. Without the docs the model invents config filenames and knob names; with the tuning-knob descriptions retrieved, it emits the exact `knob = value` in `tuning.json` that the user needs. One caveat applies: the docs condition also adds an instruction to explain rather than act, which accounts for part of the repair of answers that fired a directive instead of explaining; the identifier-level repair (the exact knob, file, and default) requires the retrieved text (Appendix A).

Question type	No memory	+ Docs memory
Knowledge (how-do-I)	58% (30/52)	65% (34/52)
Reconfigure	17% (5/30)	**93% (28/30)**
Overall (n=82)	43% (35/82)	**76% (62/82)**

Table 3: Self-knowledge and self-reconfiguration (claude-haiku-4-5, judged by claude-sonnet-5). Docs memory helps knowledge modestly but is decisive on reconfiguration, where retrieved knob descriptions supply the exact file, name, and default.

6 Conclusions and Future Work

We described MARS, a single-binary terminal editor, multiplexer, and agent with memory over the user's own local context. In the terminal's constrained domain, lexical retrieval over local files raised a small model from 38% to 96% on personalized translation and from 17% to 93% on self-reconfiguration. We read this as the first rung of a ladder. Above single-command recall lies *skill learning*: mining repeated command sequences into reusable procedures. Above that lies *workflow understanding*: recognizing a training run or debug session in a command stream, predicting the next step, and flagging deviation. Furthest out is *cross-fleet memory*: semantic retrieval once paraphrase matters, and memory that follows the user across machines through the same key broker. The through-line is ownership: local, private, portable, compounding memory, which an assistant that sees only what it is handed cannot replicate. A cloud assistant starts every conversation a stranger; an agent that owns its user's memory becomes a little less generic, and a little less amnesiac, with every correction. Simple memory works surprisingly well; the same substrate, enriched, points to agents that learn your skills and understand your work.

Limitations

Four limitations bound these results. Firstly, the gold sets, though frozen before any run, are modest (112 and 82 items), so the personalized results should be read as evidence that memory closes this gap on this set, not as a claim at scale. Secondly, only one model is under test, so the numbers characterize the small, inexpensive regime MARS targets rather than models broadly; whether paraphrase generalization holds for a larger or reasoning model is an open question. Thirdly, the judge (claude-sonnet-5) is stronger than the model under test but comes from the same model family, so a family-level preference cannot be ruled out, and an LLM judge carries noise in any case; the appendix quantifies the observed slack at a few points in either direction. Finally, the corrective-memory store contained only the 24 seeded pairs during the experiment. A long-lived store holds hundreds of entries, and although the terminal domain's lexical stability favors BM25 (§3.6), we have not measured retrieval precision against a realistic population of distractors.

Ethics Statement

MARS reads the user's shell history (`$HISTFILE`) and stores accepted commands in a plaintext local file (`~/.mars/cmd_memory`). Shell history can contain secrets: tokens pasted into commands, passwords embedded in URLs, internal hostnames. Retrieval can therefore inject such strings into a prompt bound for a cloud provider. The store is local, human-readable, and user-editable, and nothing leaves the machine except the prompt of the current request, but MARS does not currently redact retrieved entries; a redaction pass and a per-entry denylist before prompt injection are planned, and users who work with sensitive history can point MARS at a local model, which keeps prompts on the machine entirely. On the credential side, the key broker (§4) is itself a mitigation: it removes the incentive to scatter API keys across remote machines. The evaluation involved no human subjects and no data beyond author-written gold sets and the authors' own usage logs.

References

Anthropic. 2024. Introducing computer use. https://www.anthropic.com/news/3-5-models-and-computer-use. Accessed July 2026.

Anthropic. 2025. Claude code. https://www.anthropic.com/claude-code. Accessed July 2026.

Anysphere. 2024. Cursor: The AI code editor. https://cursor.com. Accessed July 2026.

Lutfi Eren Erdogan, Nicholas Lee, Sehoon Kim, Suhong Moon, Hiroki Furuta, Gopala Anumanchipalli, Kurt Keutzer, and Amir Gholami. 2025. Plan-and-act: Improving planning of agents for long-horizon tasks. arXiv:2503.09572.

GitHub. 2024. GitHub Copilot in the CLI. https://docs.github.com/en/copilot/github-copilot-in-the-cli. Accessed July 2026.

Ellie Huxtable and Atuin contributors. 2024. Atuin: Magical shell history. https://atuin.sh. Accessed July 2026.

Vladimir Iakovlev and contributors. 2024. The fuck: Corrects your previous console command. https://github.com/nvbn/thefuck. Accessed July 2026.

Anjishnu Kumar. 2026. MARS: A memory-augmented agentic terminal multiplexer. https://crates.io/crates/mars-terminal. Software, installable via `cargo install mars-terminal`. Accessed July 2026.

Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. 2020. Retrieval-augmented generation for knowledge-intensive NLP tasks. In *Advances in Neural Information Processing Systems (NeurIPS)*.

Xi Victoria Lin, Chenglong Wang, Deric Pang, Kevin Vu, Luke Zettlemoyer, and Michael D. Ernst. 2017. Program synthesis from natural language using recurrent neural networks. *University of Washington Department of Computer Science and Engineering, Tech. Rep. UW-CSE-17-03-01*.

Xi Victoria Lin, Chenglong Wang, Luke Zettlemoyer, and Michael D. Ernst. 2018. NL2Bash: A corpus and semantic parser for natural language interface to the linux operating system. In *Proceedings of the Eleventh International Conference on Language Resources and Evaluation (LREC)*.

Nicholas Marriott and tmux contributors. 2024. tmux: a terminal multiplexer. https://github.com/tmux/tmux. Accessed July 2026.

Donald A. Norman. 1988. *The Psychology of Everyday Things*. Basic Books.

Stephen Robertson and Hugo Zaragoza. 2009. The probabilistic relevance framework: BM25 and beyond. *Foundations and Trends in Information Retrieval*, 3(4):333–389.

Timo Schick, Jane Dwivedi-Yu, Roberto Dessì, Roberta Raileanu, Maria Lomeli, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. 2023. Toolformer: Language models can teach themselves to use tools. In *Advances in Neural Information Processing Systems (NeurIPS)*.

Warp. 2024. Warp: The intelligent terminal. https://www.warp.dev. Accessed July 2026.

Zhiyong Wu, Zhenyu Wu, Fangzhi Xu, Yian Wang, Qiushi Sun, Chengyou Jia, Kanzhi Cheng, Zichen Ding, Liheng Chen, Paul Pu Liang, and Yu Qiao. 2025. OS-ATLAS: A foundation action model for generalist GUI agents. In *International Conference on Learning Representations (ICLR)*.

Tianbao Xie, Danyang Zhang, Jixuan Chen, Xiaochuan Li, Siheng Zhao, Ruisheng Cao, Toh Jing Hua, Zhoujun Cheng, Dongchan Shin, Fangyu Lei, et al. 2024. OSWorld: Benchmarking multimodal agents for open-ended tasks in real computer environments. In *Advances in Neural Information Processing Systems (NeurIPS), Datasets and Benchmarks Track*.

Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. 2023. ReAct: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations (ICLR)*.

Zellij contributors. 2024. Zellij: a terminal workspace and multiplexer. https://zellij.dev. Accessed July 2026.

Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Siyuan Zhuang, Zhanghao Wu, Yonghao Zhuang, Zi Lin, Zhuohan Li, Dacheng Li, Eric P. Xing, Hao Zhang, Joseph E. Gonzalez, and Ion Stoica. 2023. Judging LLM-as-a-judge with MT-bench and chatbot arena. In *Advances in Neural Information Processing Systems (NeurIPS), Datasets and Benchmarks Track*.

A Qualitative Examples and Error Analysis

Every example in this appendix is quoted verbatim from the frozen result files the evaluation was scored on: `axis_a.jsonl` (112 translation items), `lift.jsonl` (24 leakage-controlled cold/paraphrase/verbatim items), and `axis_b.jsonl` (self-knowledge, 164 rows over the two memory conditions), with the model under test claude-haiku-4-5 and the judge claude-sonnet-5 throughout. We organize the analysis by experiment, and within each by impressive behavior, unexpected errors, phenomena worth naming, and concrete recommendations. The purpose is to show, from the raw data, exactly where a simple local-memory mechanism helps, where it does not, and where the measurement itself is the limiting factor.

A.1 Axis A: NL→shell translation

Impressive behavior. On general commands the small model is a competent shell author, and its correct outputs are not only the easy one-liners: it composes multi-stage pipelines with the right utility at each stage, including summation and column projection in `awk`, buffered streaming, and process substitution. These were all judged correct:

```
a20  find the ten largest files under the cwd
   -> find . -type f -printf '%s %p\n' \
        | sort -rn | head -10
a66  count total lines across all python files
   -> find . -name "*.py" -type f -exec wc -l {} + \
        | tail -1
a07  kill the process listening on port 8080
   -> lsof -ti:8080 | xargs kill -9
a30  count how many times each unique line appears
   -> sort | uniq -c
a80  print columns 1 and 3 of a comma-separated file
   -> awk -F',' '{print $1,$3}' filename
```

This is the premise of the paper made concrete: general shell translation is a near-solved commodity even on a small, inexpensive model (79 of 88 general items correct), so the accuracy that matters is the personalized slice, which the model cannot know from the request text alone.

Unexpected errors. Two error classes account for the general-command failures, and, unlike the free-tier runs of §5.1, this run produced zero empty completions, so every failure is a real answer rather than a token-wall blank. The first class is *placeholder and precision misses*: the model guesses a stand-in filename that the gold set spells differently (`a69` "print the contents of a file with line numbers" → `cat -n`, gold `cat -n file.txt`; `a31` `grep error log file` vs. `grep error app.log`), or picks a plausible but semantically different tool (`a38` "show my current IP" → `ip addr show`, a LAN address, where the reference wants `curl ifconfig.me`; `a86` reads from `/dev/stdin` where the gold splits a file into words). The second class dominates the personalized slice: *ecosystem blindness*, in which the model answers a Rust or `uv` project with the statistically commonest ecosystem: `p04` "run the unit tests" → `npm test` (gold `cargo test`), `p06` "lint the code" → `eslint .` (gold `ruff check .`), `p11` "build the release binary" → `make release` (gold `cargo build --release`), `p18` "install project dependencies" → `npm install` (gold `uv sync`). A milder personalized pattern is *honest abstention*: on the hardest project commands the model declines rather than bluff (`p03` "deploy the model,"

p12 "evaluate the latest checkpoint" → "I need more context…"), which is safe behavior but still scored wrong against a gold command. Both classes are unknowable from the request text, and both are exactly what corrective memory targets.

Phenomena. Even under the stronger claude-sonnet-5 oracle, the judge assigns a *partial* band that captures near-misses, and reading it is informative: the general failures split into 6 outright wrong and 3 partial, and the partials are precisely the placeholder-filename cases (a22 `chmod +x script` vs. gold `script.sh`; a77 `zip -r archive.zip .` vs. `. . . folder/`) where the model understood the task but guessed a different stand-in. The same slack explains why the two no-memory measurements of the 24 personalized items disagree: the base row of Table 2 (42%, 10/24) and the lift experiment's cold condition (38%, 9/24) come from separate runs, and they differ on five items. Three differ because the cold completions were regenerated and the model produced a different command (p10, p14, p17); two differ because the judge graded the identical output differently across runs (p01 `source venv/bin/activate`, correct vs. partial; p22 `docker push`, wrong vs. correct). The disagreements move in both directions and net to one item. The practical consequence is that every point estimate in this paper carries a couple of points of generation-and-judging slack, and we read the results through that band.

Recommendations. Two inexpensive fixes would tighten the measurement without changing the system's thesis. Firstly, placeholder-normalized scoring, or explicit partial credit, so that a filename stand-in is not counted as a comprehension error; the 6 cold *partials* of the lift set (below) show how much signal lives in this band. Secondly, marking honest abstentions distinctly from wrong guesses, since "I need more context" on a personalized command is a different failure mode from confidently emitting the wrong ecosystem. Neither fix touches the model or the memory mechanism; both are evaluation hygiene.

A.2 Corrective memory

Impressive behavior. This is the headline result, and the per-item data shows the mechanism cleanly, under the leakage-controlled protocol in which the store holds only a *paraphrase* of each prior request (mean word-Jaccard 0.36 to the test query). With those reworded exemplars retrieved by BM25 into the prompt, 23 of 24 project-specific items are correct, up from 9 correct cold: 15 items flip from wrong-or-partial to correct, and one regresses (below). Table 4 quotes representative flips verbatim. Two sub-mechanisms are visible. The first is *ecosystem correction*: a Rust or uv-native project answered with the commonest default (`npm`, `eslint`, `shellcheck`) is corrected to the user's real toolchain. The second is *exact-argument recall*: the model already had the right utility but the wrong specifics, and memory supplies the precise config, path, or split the user expects reproduced (p02 "run the training job" gains `–config configs/base.yaml`; p14 "run inference on the test set" becomes `python infer.py –split test`).

Unexpected errors. The paraphrase column has a single error out of 24, and it is also the experiment's one *regression*: an item that was correct cold became wrong with memory, which is why 9 cold-correct plus 15 flips yields 23 warm-correct rather than 24. Cold, p19 "run the benchmark suite" was answered `make benchmark`, which the judge accepted as a plausible route to the gold `python -m benchmarks.run`. With the paraphrased exemplar retrieved, the model instead answered `cargo bench`: it resolved the ecosystem to Rust, and the retriever did not surface the paraphrased Python exemplar strongly enough to override that prior. This is a retrieval-plus-prior miss, not a lookup failure, and it is the honest residual of a method that must generalize rather than copy. The shape of the cold column is also informative: of the 15 non-correct items, 9 are outright wrong and 6 partial, so memory is repairing a mix of fundamentally-wrong ecosystem guesses and last-mile precision misses, not merely polishing near-correct commands.

Phenomena. The gap between the two warm conditions is the cleanest measure of what BM25 buys here. Verbatim memory (the exact prior request in the store) scores 100% (24/24), the trivial lookup ceiling, while paraphrase memory scores 96% (23/24). The paraphrase condition falls only one item short of verbatim lookup despite a mean word-Jaccard of just 0.36 between the stored exemplar and the query: the retriever bridges a substantial rewording gap on 23 of 24 items, so the personalization win is a genuine generalization result rather than an artifact of a planted answer. The

Request	Cold (no memory)	+ Paraphrase memory
run the unit tests	`npm test`	`cargo test`
lint the code	`eslint .`	`ruff check .`
build the release binary	`make release`	`cargo build --release`
install project dependencies	`npm install`	`uv sync`
check code formatting	`shellcheck *.sh`	`black --check .`
clean the build artifacts	`rm -rf build/ dist/ . . .`	`cargo clean`
run the training job	`python train.py`	`python train.py --config configs/base.yaml`
deploy the model	*(asks for more context)*	`./scripts/deploy.sh --prod`

Table 4: Representative wrong/partial→correct flips from the leakage-controlled experiment (`lift.jsonl`). The store held only a *paraphrase* of each prior request (mean word-Jaccard 0.36) and the model was tested on the original, so a warm win requires generalization across the paraphrase gap, not lookup. The top block shows *ecosystem correction* (a Rust/uv project answered with `npm`/`eslint`/`shellcheck` defaults); the last two rows show *exact-argument recall* and *abstention rescue*. All warm outputs match gold.

one-item gap is the price of that generalization.

Recommendations. The residual is a *retrieval* miss, not a generation miss: when the paraphrase drifts far enough and a strong ecosystem prior competes, lexical BM25 does not surface the stored pair decisively. The cheapest next lever, given that stored commands now carry `ts`/`session`/`cwd` fields, is recency-and-project weighting: when several exemplars tie lexically, prefer the user's recent and same-directory commands. Semantic retrieval with embeddings is the rung after that, and is warranted only once the domain broadens enough that the paraphrase gap dominates (§6).

A note on token cost. Memory's effect on token cost is model-dependent, an observation that cuts against a claim an earlier draft of this work made. On non-reasoning claude-haiku-4-5, retrieved exemplars add a small input cost (mean 95→161 tokens per call, +66). On a *reasoning* model (Qwen3-32B, from the free-tier runs of §5.1), however, the same few-shot grounding suppresses the verbose chain-of-thought the model otherwise emits when unsure, so net cost falls (385→217 per call, −44%). Memory as a token *reduction* is therefore a reasoning-model phenomenon, not a universal property; we report the per-model effect rather than headline the favorable case.

A.3 Axis B: self-knowledge

Docs memory lifts overall accuracy from 43% to 76% (35→62 of 82), but the two question types behave very differently, and the split between them is informative. Retrieval turns a plausible guess into a grounded fact:

```
k02 "How do I save the current file?"
 none: "Use the Save action."
      [would run: Save]            (acts, no key)
 docs: "Press C-x C-s to save
        the current buffer."         (correct)
r16 "How do I change the accent color?"
 none: "Mars doesn't have built-in
        theme color settings ..."    (wrong)
 docs: set theme_accent to an [r,g,b]
       array in tuning.json          (correct)
```

The reconfigure slice is where memory is decisive, rising from 17% to 93% (5→28 of 30). Without docs the model invents plausible config locations: `r16` "change the accent color" is answered "Mars doesn't have built-in theme color settings," and `r04` "use a different model" points at a `~/.mars/config.json` that does not exist. With the retrieved knob descriptions, which carry the exact name, file, and default, the agent answers `theme_accent` / `MARS_LLM_MODEL` in `tuning.json` correctly. Two failure families that motivated the corpus design, *acts-instead-of-explains* (answering a how-to question with a bare `[would run: . . .]` directive, `k02`, `k09`) and *hallucinated identifiers* (a right-idea knob under the wrong filename), are largely eliminated on this slice by the actionable, path-carrying knob descriptions and an explain-don't-act instruction on the docs path.

Knowledge questions gain far less, from 58% to 65%, for an instructive reason: they turn on an *exact keybinding*, and even with the docs retrieved the model sometimes prefers a plausible chord over the one the docs state (`k05` "detach my session" → `C-x C-c`, gold `C-x C-d`; `k06` "search within a buffer" → `C-f`, gold `C-s`; `k18` "jump to top/bottom" → `C-x Home`, gold `M-<`). A knob *name* is a token the retriever can hand over intact; a chord is a specific fact the model will still overwrite with a more common default, which is why

doc memory helps reconfiguration far more than keybinding recall. A residual limit is judge slack: a few correct-but-differently-phrased answers are marked wrong, so the reported knowledge number is a mild lower bound even under the stronger claude-sonnet-5 oracle.